

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

**Duration : 1 Hour
Total Marks:30**

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____P.NAGENDRA REDDY(192472399)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NAGENDRA REDDY

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS

1.

Backtracking Search: Doctor Shift Assignment

1. Problem Definition

A hospital must assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) using Backtracking Search while satisfying all the constraints.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3: D2's shift must be earlier than D3's shift.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

2. Step 1: Define the Variables and Domains

- Variables: D1, D2, D3
- Domains: Morning (M), Afternoon (A), Night (N)

The assignments must satisfy:

- $(D1 \neq N)$
- $(D2 < D3)$, where $(M < A < N)$
- $(D3 \neq M)$
- All shifts must be different.

4. Step 2: Apply Backtracking Search

We assign shifts one at a time and check the constraints after each assignment. If a constraint is violated, we backtrack and try another shift.

Intermediate steps

Step 1: Assign D1 = Morning

Valid

- D1 is not assigned Night.
- No other doctor has been assigned yet.

Step 2: Try D2 = Afternoon

Valid

- D1 = Morning and D2 = Afternoon are different.
- D2 can work before D3 if D3 is assigned Night.

Step 3: Try D3 = Morning

Invalid

- Violates constraint (iv): D3 cannot work Morning.
- Backtrack and try another shift.

Step 4: Try D3 = Afternoon

Invalid

- Afternoon is already assigned to D2.
- Violates constraint (iii): only one doctor per shift.
- Backtrack and try Night.

Step 5: Try D3 = Night

Valid

- D3 is not assigned Morning.
- D2 works Afternoon before D3 works Night.
- All shifts are assigned to different doctors.

5. Step 3: Final Constraint Verification

D1 cannot work Night	D1 = Morning	Satisfied
D2 must work before D3	Afternoon < Night	Satisfied
Only one doctor per shift	All shifts are different	Satisfied
D3 cannot work Morning	D3 = Night	Satisfied
Every doctor gets exactly one shift	D1, D2, D3 each assigned once	Satisfied

5. Final Valid Schedule

Final answer: D1 → Morning, D2 → Afternoon, D3 → Night.

Conclusion:

Backtracking Search finds a valid schedule by assigning shifts sequentially, checking constraints, and reversing invalid choices until all constraints are satisfied.

2.

1. Robot Navigation in a 5×5 Grid Using BFS and Manhattan Distance

A. Problem definition

A robot must move from the Start (S) to the Goal (G) in a 5×5 grid. Obstacles block certain cells, and the robot can move only Up, Down, Left, or Right.

Given constraints:

- Grid size: 5×5
- Movement: Up, Down, Left, Right
- Cost of each move: 1

- Obstacles: Cannot be traversed
- Heuristic: Manhattan distance

Manhattan distance is calculated as:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

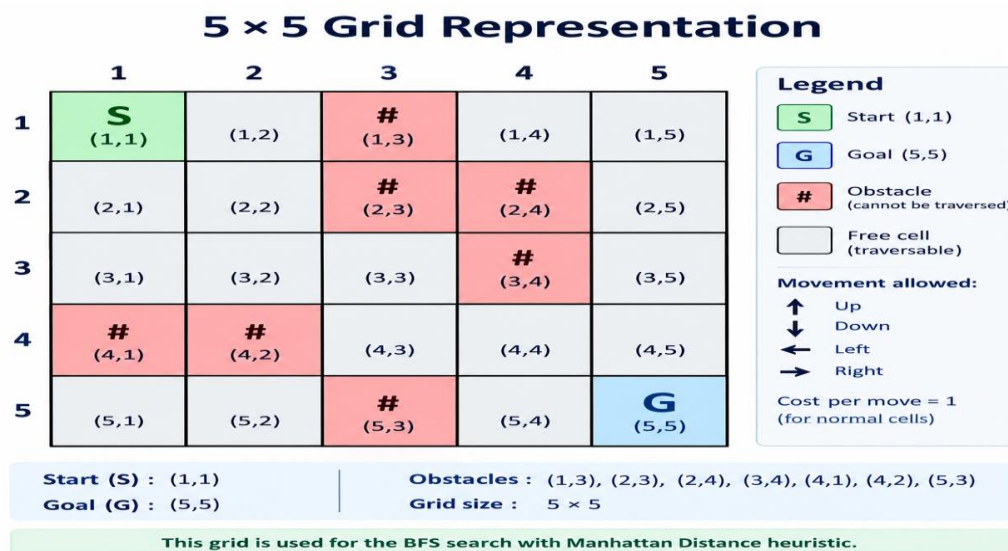
Here, (x_n, y_n) is the current cell and (x_G, y_G) is the goal cell.

Important distinction: BFS (Breadth-First Search) uses a FIFO queue and finds the shortest path when all moves have equal cost. It does not use the Manhattan heuristic to order nodes. If the heuristic must determine which node to expand next, A* search is appropriate.

Since the question asks for BFS, we will use BFS to find the optimal path and show the Manhattan distance as additional information.

B. Grid representation

As the exact obstacle positions were not specified, consider this example grid. # represents an obstacle.



Coordinates are represented as (row, column), starting from (1,1).

- Start: $(S=(1,1))$
- Goal: $(G=(5,5))$
- Obstacles: (1,3), (2,3), (2,4), (3,4), (4,1), (4,2), (5,3)

The Manhattan distance from the start to the goal is:

$$h(S) = |1-5| + |1-5| = 8$$

This is a lower bound on the number of moves required, ignoring obstacles.

C. Step-by-step BFS node expansion and queue updates

BFS maintains a FIFO queue. The first node inserted is the first node expanded. We mark cells as visited when they are added to the queue to avoid duplicates.

The queue below is shown after each node expansion, using the neighbor order Up → Down → Left → Right.

Step	Expanded node	New valid nodes added	Queue after expansion
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(1,2), (3,1), (2,2)
3	(1,2)	None	(3,1), (2,2)
4	(3,1)	(3,2)	(2,2), (3,2)
5	(2,2)	None	(3,2)
6	(3,2)	(3,3)	(3,3)
7	(3,3)	(4,3)	(4,3)
8	(4,3)	(4,4)	(4,4)
9	(4,4)	(5,4), (4,5)	(5,4), (4,5)
10	(5,4)	(5,5)	(4,5), (5,5)
11	(4,5)	(3,5)	(5,5), (3,5)
			Stop
12	(5,5)	Goal reached	

When the goal is dequeued at step 12, the algorithm terminates. The goal's parent points to (5,4), allowing us to reconstruct the shortest path

E. Optimal path and total cost

Shortest path found by BFS

S (1,1) ↓ (2,1) ↓ (3,1) → (3,2) → (3,3) ↓ (4,3) → (4,4) ↓ (5,4) → G (5,5)

Total moves

Shortest path found by BFS	
S (1,1)	
↓	
(2,1)	
↓	
(3,1) → (3,2) → (3,3)	
↓	
(4,3) → (4,4)	
↓	
(5,4) → G (5,5)	
Total moves	8
Cost per move	1
Total path cost	8

Conclusion:

BFS finds a shortest path of cost 8 for the example grid. The Manhattan heuristic estimates the remaining distance but does not influence BFS queue ordering.

3.

2. Autonomous Rescue Robot Using Uniform Cost Search and Online Search

A. Analyze the constraints

The rescue robot must reach a survivor in a disaster-affected building. Because it can observe only adjacent cells, it cannot know the entire environment in advance.

Constraint	Effect on robot decision-making
Movement in four directions	The robot explores Up, Down, Left, and Right.
Obstacles	Blocked cells are excluded from possible movements.
Limited sensing	The robot discovers obstacles and paths as it moves.
Movement cost = 1	Every normal movement adds 1 to the total cost.
Risky-zone penalty = 2	Entering a risky cell costs $(1+2=3)$.
Minimize total cost	The robot chooses a low-cost safe route rather than simply the route with the fewest steps.
Dynamic knowledge	The robot updates its map and replans when new information becomes available.

Key point:

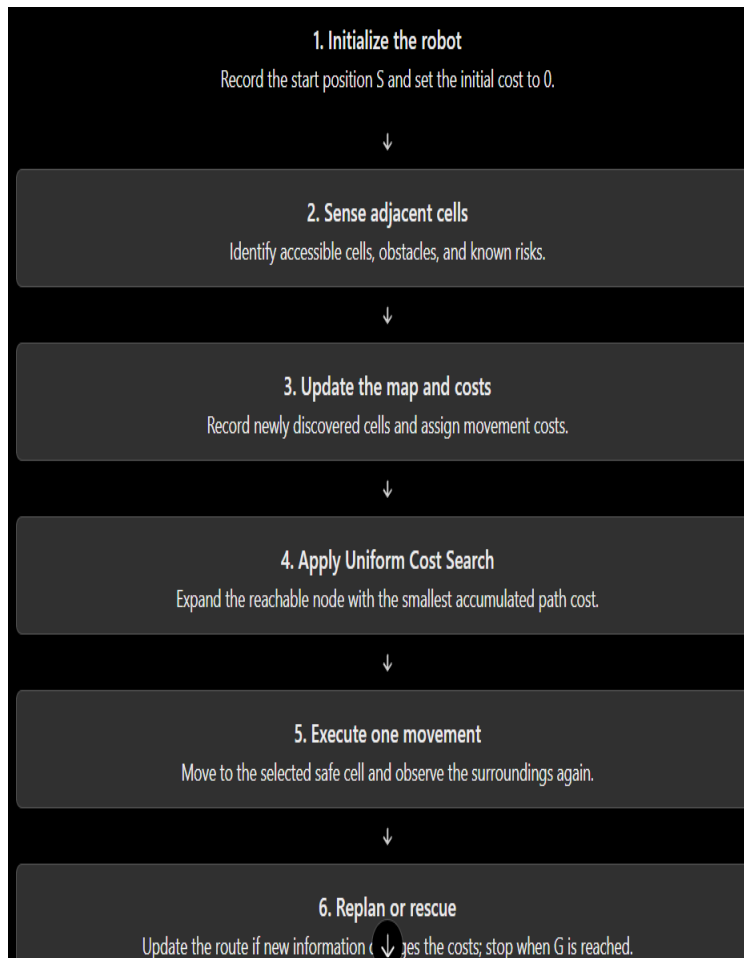
The shortest path in distance is not necessarily the lowest-cost path. A longer route may be preferable if it avoids risky areas.

C. Proposed search strategy

Use a combination of Uniform Cost Search (UCS) and an Online Search Agent.

- UCS selects the available path with the smallest accumulated cost.
- The online agent discovers the environment incrementally and replans as it learns about obstacles and risks.

D. Step-by-step working procedure



Illustrative cost and priority queue updates

Suppose the robot has discovered these possible routes from its current location.

Candidate route	Movement costs	Total cost
Route A	Normal, Normal, Risky	$\sqrt{(1+1+3=5)}$
Route B	Normal, Normal, Normal, Normal	$\sqrt{(1+1+1+1=4)}$
Route C	Normal, Risky	$\sqrt{(1+3=4)}$

The priority queue is ordered by accumulated cost:

Queue priority	Candidate	Cost
1 (tie)	Route B	4
1 (tie)	Route C	4
3	Route A	5

The robot expands a lowest-cost candidate first. When two routes have equal cost, it may use a tie-breaking rule, such as preferring the route with fewer risky cells.

These routes are illustrative examples, not a computed solution for a specific building layout. The exact optimal rescue path cannot be calculated without the actual obstacle and risk map.

D. Application of AI concepts

1. Intelligent agents

The robot acts as an intelligent agent because it senses the environment, processes information, selects an action, and observes the result.

- Sensors: Detect adjacent walls, obstacles, hazards, and accessible paths.
- Agent program: Maintains the map and applies UCS.
- Actuators: Move the robot through the building.
- Goal: Reach the survivor at minimum cost while maintaining safety.

2. Rationality

A rational agent selects actions that best achieve its objective using the information currently available.

For example, if a direct route passes through a risky cell, the robot compares the added risk cost with the cost of an alternative route. It selects the lowest-cost route among the currently known safe alternatives.

Rationality does not mean the robot always knows the globally optimal route immediately. It means it makes the best justified

decision using its available information and updates that decision when new evidence arrives.

3. Environment type

Environment property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed.
Determinism	Potentially stochastic	Hazards or movement outcomes may be uncertain. If all outcomes are fixed, it is deterministic.
Dynamics	Dynamic	New obstacles or hazards may appear.
State space	Discrete	The building consists of grid cells.

Actions	Sequential	Each movement affects the next decision.
Knowledge	Initially unknown	The robot learns the map while moving.
Agents	Usually single-agent	One robot plans the route, unless other autonomous agents are introduced.

E. Why UCS with online search is suitable

1. Cost optimization: UCS accounts for both movement costs and risky-zone penalties.
2. Obstacle avoidance: The robot excludes known blocked cells from its search.
3. Limited information: Online exploration allows the robot to discover the building gradually.
4. Dynamic replanning: The robot can update its map and recompute a route after discovering a hazard.
5. Optimality: UCS finds a minimum-cost path when costs are nonnegative and the relevant state space is sufficiently explored. In an online environment, global optimality cannot be guaranteed if important parts of the map remain unknown or change unexpectedly.

6. Safety: The robot can reject known hazardous or blocked cells and use risk-aware costs to select safer alternatives.

A practical implementation should also enforce hard safety rules for unacceptable hazards. A penalty of 2 alone does not guarantee that the robot will avoid every dangerous area.

Final conclusion

For the first problem, BFS identifies a shortest path of 8 moves with a total cost of 8 for the illustrative 5×5 grid.

For the rescue robot, UCS combined with an online search agent is a suitable approach because it minimizes accumulated movement and risk costs while allowing the robot to sense, learn, update its map, and replan. The actual rescue route and its total cost depend on the building's obstacle and risk layout.